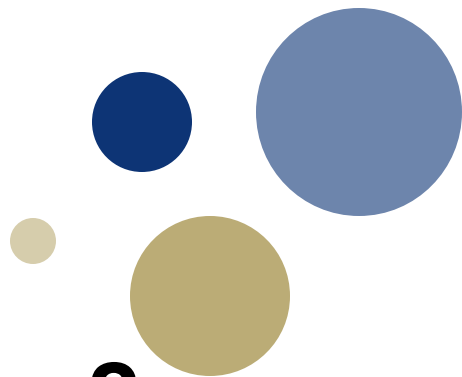




NTNU

Kunnskap for en bedre verden

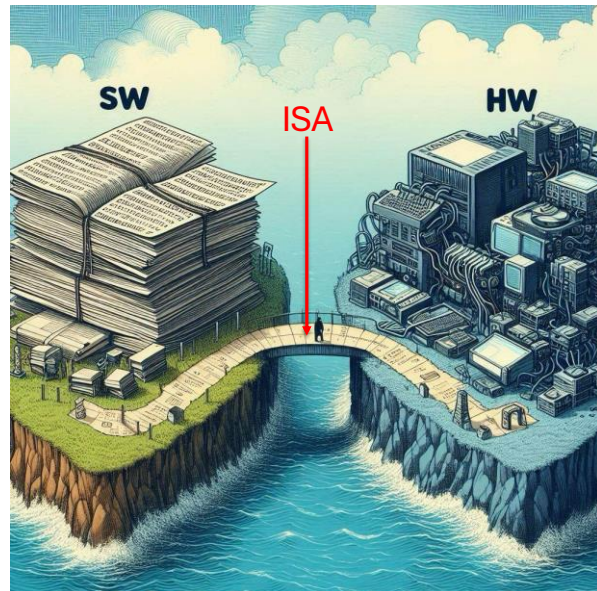


Forelesning 3: Instruksjoner 2

TDT4160 Datamaskiner

Instruksjonssett

- «Instruction Set Architecture (ISA)»
- Grensesnittet mellom programvare og maskinvare
- Vi vil at maskinen skal kunne beregne alle beregnbare funksjoner (være Turing-komplett), og da trenger vi:
 - Aritmetiske instruksjoner (add, addi)
 - Minneinstruksjoner (lw og sw)
 - Instruksjoner som tar beslutninger



Laget av Lasse Natvig med “NTNU
Copilot” designer



Tema 2.1: Instruksjoner (Kapittel 2.5)

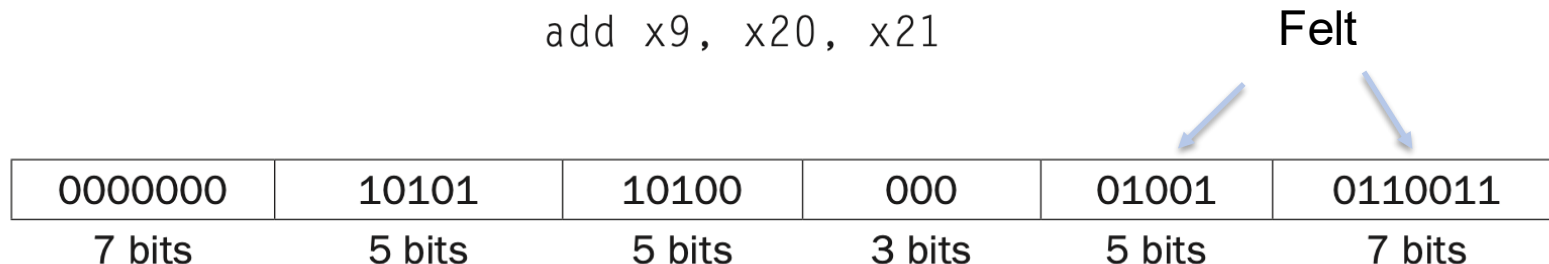
HVORDAN REPRESENTERES INSTRUKSJONER?

Læringsutbytte T2.1

- Studenten kjenner til de tre **designprinsippene** for instruksjonssettdesign og deres motivasjon.
- Studenten kan **oversette** fra et **høynivåspråk** til **assemblyinstruksjoner** (og omvendt).
- Studenten kan oversette fra **assemblyinstruksjoner** til **maskinkode** (og omvendt).
- Studenten kjenner til **instruksjonsformatene** i RISC-V og kan forklare hvorfor de er definert slik de er.
- Studenten forstår hvordan instruksjoner lagres i minnet og hvordan dette utnyttes til å implementere **kontrollflyt**.

Hva er en instruksjon?

- En instruksjon er, overraskende nok, en streng med 1ere og 0ere (altså på eller av)
 - ... og for at maskinen skal kunne tolke denne strengen må den ha et bestemt format



Vi kaller den numeriske versjonen av instruksjonen for maskinkode

R-type instruksjoner

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Forklaring:
 - Opcode: Operasjonskode. Angir hvilken operasjon som skal utføres
 - funct3 og funct7: Flere operasjonskoder
 - rd: Nummeret til destinasjonsregisteret
 - rs1: Nummeret til det første operandregisteret
 - rs2: Nummeret til det andre operandregisteret
- Vi kan adressere 32 registre totalt (fordi $2^5 = 32$)

Instruksjonsformat

Designprinsipp 3:
Gode løsninger krever
gode kompromiss

- Noen problemer:
 - Vi har sett at konstanter er vanlige, men R-type instruksjonene har ikke plass til konstanter
 - Instruksjonene blir enkle å dekode hvis alle instruksjoner er like lange og alle har samme format
- Løsning: Vi legger til flere instruksjonsformat men holder dem så like som mulig

I-type

F. eks. `addi` og `lw`

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

S-type

F. eks. `sw`

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Koding av instruksjoner (1)

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011
Instruction	Format	immediate		rs1	funct3	rd	opcode
addi (add immediate)	I	constant		reg	000	reg	0010011
lw (load word)	I	address		reg	010	reg	0000011
Instruction	Format	immed-iate	rs2	rs1	funct3	immed-iate	opcode
sw (store word)	S	address	reg	reg	010	address	0100011

Instruksjonsformatet er gitt av opcode og opcode er alltid på samme sted

Koding av instruksjoner (2)



R-type Instructions	funct7	rs2	rs1	funct3	rd	opcode	Example
add (add)	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sub)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3
I-type Instructions	immediate		rs1	funct3	rd	opcode	Example
addi (add immediate)	001111101000		00010	000	00001	0010011	addi x1, x2, 1000
lw (load word)	001111101000		00010	010	00001	0000011	lw x1, 1000 (x2)
S-type Instructions	immed-iate	rs2	rs1	funct3	immed-iate	opcode	Example
sw (store word)	0011111	00001	00010	010	01000	0100011	sw x1, 1000(x2)

immediate er gitt i 2's komplement og kan representere både positive og negative tall

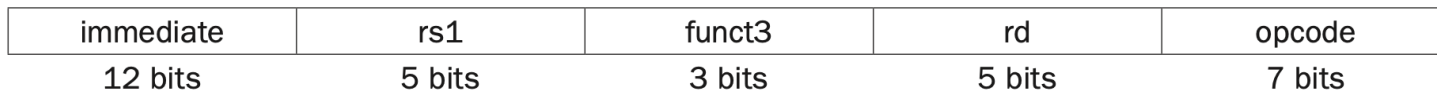
Instruksjonsformat

R-type



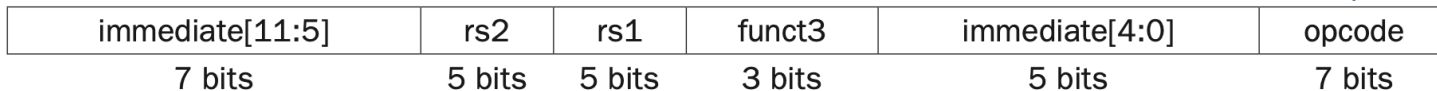
F. eks. *add*

I-type



F. eks. *addi* og *lw*

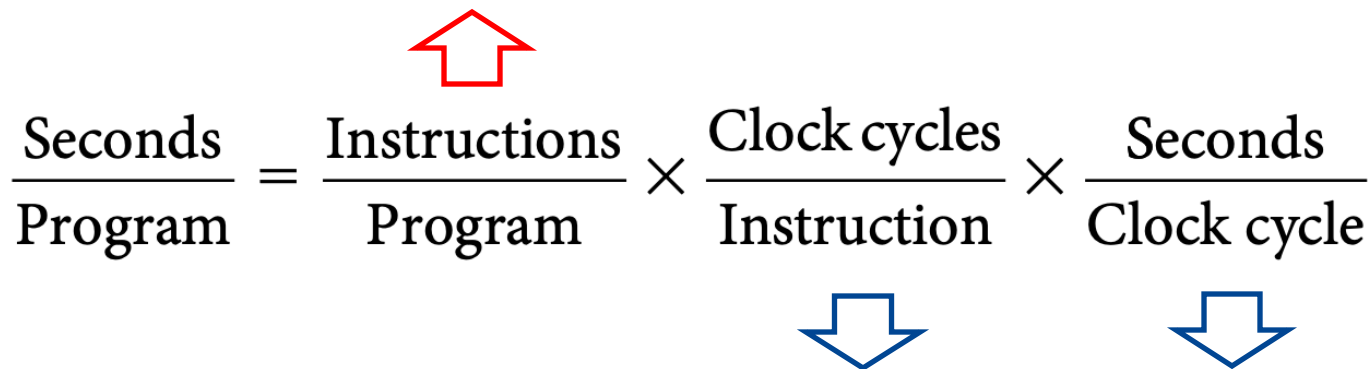
S-type



F. eks. *sw*

Feltene som betyr det samme bruker så langt det er mulig samme bit

Instruksjoner og «Iron Law»

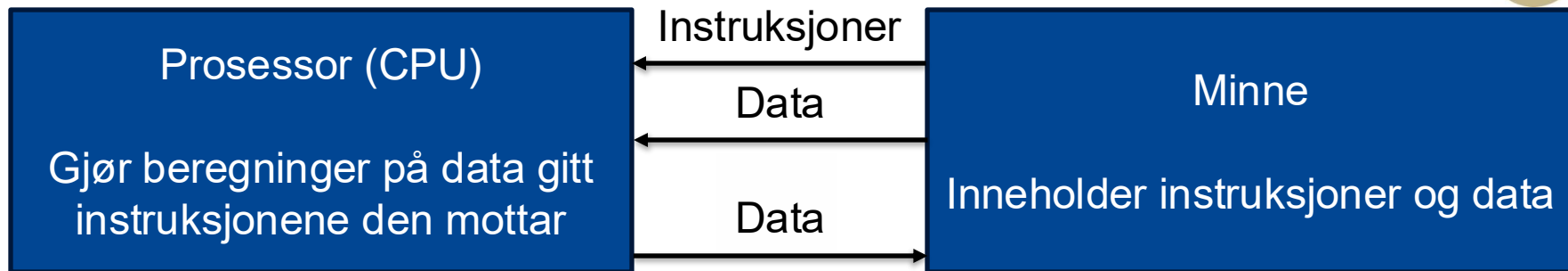

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

The diagram features a red upward-pointing arrow above the 'Instructions' term in the numerator of the second fraction, and two blue downward-pointing arrows below the 'Clock cycles' and 'Clock cycle' terms in the third fraction.

Jo likere formatene er, jo enklere blir det for maskinen å dekode instruksjonene

... men blir formatene for begrensede må vi bruke mange instruksjoner til å uttrykke programmet

Prinsippet om lagrede program



- Både instruksjoner og data er tall og det er tolkningen som avgjør om maskinen behandler tallet som det ene eller det andre
 - Maskinen må vite hvor programmet starter og så ordner det seg
- Programmer kan derfor distribueres som binærfiler til maskiner med samme instruksjonssett



Tema 2.1: Instruksjoner (Kapittel 2.7)

INSTRUKSJONER SOM TAR BESLUTNINGER

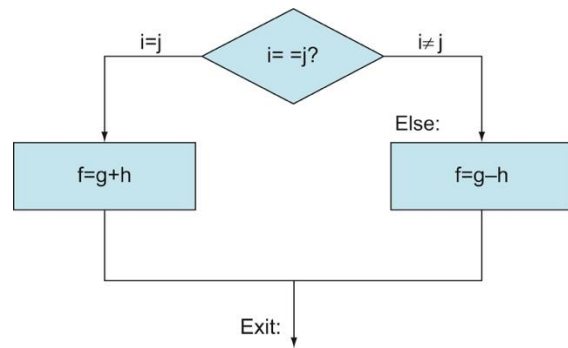
Læringsutbytte T2.1

- Studenten kjenner til de tre designprinsippene for instruksjonssettdesign og deres motivasjon.
- Studenten kan oversette fra et høynivåspråk til assemblyinstruksjoner (og omvendt).
- Studenten kan oversette fra assemblyinstruksjoner til maskinkode (og omvendt).
- Studenten kjenner til instruksjonsformatene i RISC-V og kan forklare hvorfor de er definert slik de er.
- Studenten forstår hvordan instruksjoner lagres i minnet og hvordan dette utnyttes til å implementere **kontrollflyt**.

Forgreningsinstruksjoner

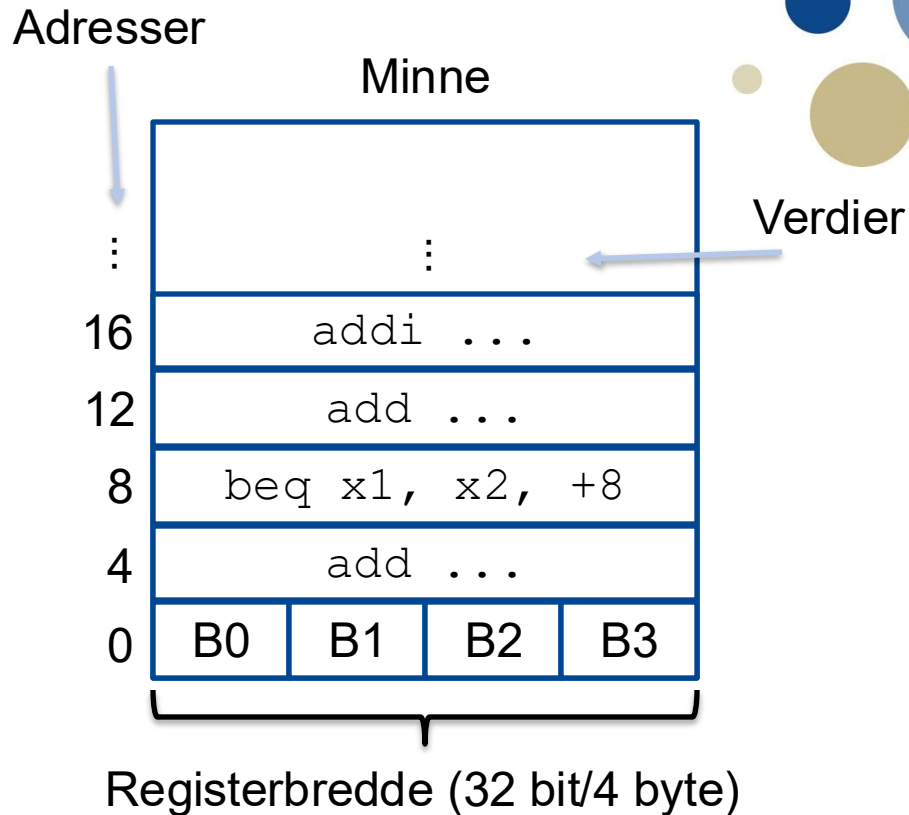
- Programmer tar ofte beslutninger basert på verdier
- Vi implementerer beslutninger med forgreningsinstruksjoner («branch instructions»)
- **Eksempel:** `beq x1, x2, merkelapp`
 - Sammenlikner verdiene i x1 og x2
 - Hvis de er like, skal vi hente neste instruksjon fra `merkelapp`
 - Hvis de er ulike, henter vi neste instruksjon som vanlig

```
if (i==j) {  
    f = g + h;  
} else {  
    f = g - h;  
}
```



Minne (igjen)

- Du kan se på minnet som en tabell 32-bit verdier («words»)
 - ... men instruksjoner er 32-bit tall!
 - Grunntilfellet er å hente neste instruksjon (Adresse + 4)
- Forgreningsinstruksjoner henter enten neste instruksjon fra adresse + 4 eller fra merkelappen
 - Merkelappen er en instruksjonsadresse (relativ til nåværende instruksjon)
- Vi bruker et spesielt register til å holde styr på adressen til den nåværende instruksjonen
 - Programteller eller «Program Counter (PC)»



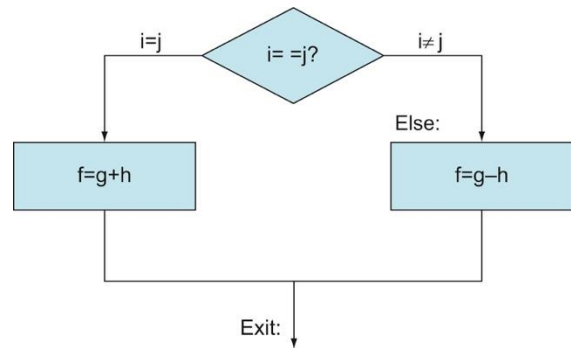
Betingede hopp

- Grovt sett finnes det to typer beslutninger:
 - Betingede («conditional»)
 - Ubetingede («unconditional»)
- Vi har så langt sett på betingede forgreningsinstruksjoner
 - ... som endrer programflyt (hopper) avhengig av verdier
- Ubetingede forgreningsinstruksjoner hopper uansett
 - Eksempel: `jal x0, merkelapp` («jump and link»)

Eksempel 1:

- Vi implementerer en beslutning i Ripes

```
if (i==j){  
    f = g+h;  
} else {  
    f = g-h;  
}
```



Eksempel 2:

- Vi implementerer en løkke i Ripes

```
int grunnverdi = 10;
for(int i=0; i<iterasjoner; i++){
    grunnverdi += 1;
}
return grunnverdi;
```





Tema 2.3: Funksjonskall (Kapittel 2.8)

FUNKSJONSKALL

Læringsutbytte T2.3

- Studenten vet hvilke **oppgaver** som skal utføres ved et **funksjonskall** og kan forklare konseptet **kallkonvensjon**.
- Studenten kjenner RISC-V **minnekartet** (kan gjengi og forklare figur 2.13).
- Studenten forstår forskjellen på **statiske** og **dynamiske data**.

Funksjonskall

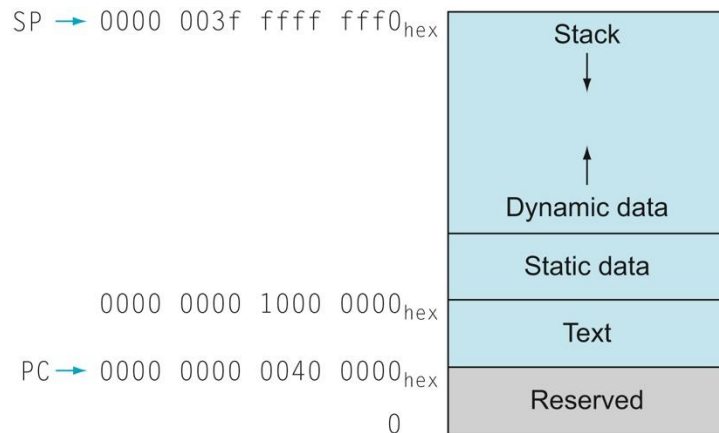
- Funksjoner hjelper oss med å strukturere koden vår
- For å kjøre en funksjon, må vi gjennom følgende oppgaver:
 1. Legg parameterne et sted funksjonen kan få tak i dem
 2. Overfør kontroll til funksjonen
 3. Alloker nødvendige ressurser (registre, minne)
 4. Utfør oppgaven
 5. Legg returverdien et sted der den som kalte funksjonen kan få tak i dem
 6. Overfør kontroll til kallstedet
- Vi vet allerede hvordan vi overfører kontroll (`jal` og `jalr`)
 - ... og grunnen til at vi må huske hvor funksjonen ble kalt fra er at funksjoner typisk kalles fra mange ulike steder i koden

Minnekart

Dynamisk = bestemt mens programmet kjører

Statisk = bestemt når programmet kompiles

- Minnekart beskriver hvordan adresserommet skal brukes:
 - Programmet lastes inn i «Text» segmentet
 - Statiske data (for eksempel globale variabler) legges i «Static data»
 - Dynamiske data (`malloc` og `new`) kalles «heap» og vokser mot høyere adresser
 - Stakken («Stack») brukes til funksjonskall og vokser mot lavere adresser
- Hvis stakken møter dynamisk allokerende data, får vi «stack overflow»
- I dette eksemplet er minnekartet regler for å få kode skrevet at ulike personer/organisasjoner til å fungere sammen
 - Kan også mappe adresser til fysiske minneenheter



Figur 2.13

Statiske data bruker stakken



Funksjonskall

1. Legg parameterne et sted funksjonen kan få tak i dem
 2. Overfør kontroll til funksjonen
 3. Alloker nødvendige resurser (registre, minne)
 4. Utfør oppgaven
 5. Legg returverdien et sted der den som kalte funksjonen kan få tak i dem
 6. Overfør kontroll til kallstedet
- Skritt 1, 3 og 5 handler alle om å allokere og deallokere statiske data
 - Vi vil helst bruke registre
 - ... men hvis vi går tom må vi bruke minne
 - ... og når vi bruker funksjoner går vi fort tom for registre

Nøstede funksjonskall

- Funksjoner kan kalle andre funksjoner (nøstede funksjonskall)
- Etter hvert funksjonskall skal **tilstanden** til datamaskinen være forutsigbar
 - Vi må vite hvilke registre som har samme verdi og hvilke som kan ha endret seg

```
a() {  
    b();  
    c();  
}
```

```
b() {  
    d();  
    e();  
}
```

```
c() {  
    // arbeid  
}
```

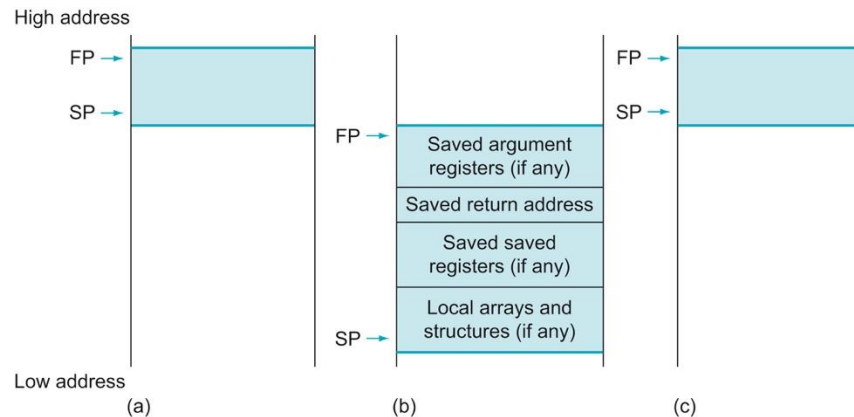
```
d() {  
    // arbeid  
}
```

```
e() {  
    // arbeid  
}
```



Løsning: Vi lagrer det vi må huske på stakken

- «Frame Pointer» (FP)
 - Peker på det første ordet i den nåværende funksjonens minneområde
- «Stack Pointer» (SP)
 - Peker på det siste ordet i den nåværende funksjonens minneområde
- Vi adresserer disse verdiene som konstante avstander til SP eller FP
 - Minneområdet under SP brukes typisk til funksjonens lokale variabler



Kallkonvensjon

- For å sikre interoperabilitet mellom kompilatorer, bibliotek, etc. må vi ha regler for hvilken tilstand som bevares etter funksjonskall
- Vi har så langt brukt de fysiske navnene til registrene (x0-x31)
 - De har også logiske navn gitt av hvordan de skal brukes
 - x5-x7 kan for eksempel refereres til som t0-t2 (t = «temporary»)

Preserved	Not preserved
Saved registers: x8-x9, x18-x27	Temporary registers: x5-x7, x28-x31
Stack pointer register: x2(sp)	Argument/result registers: x10-x17
Frame pointer: x8(fp)	
Return address: x1(ra)	
Stack above the stack pointer	Stack below the stack pointer

Name	Register number	Usage	Preserved on call?
x0	0	The constant value 0	n.a.
x1 (ra)	1	Return address (link register)	yes
x2 (sp)	2	Stack pointer	yes
x3 (gp)	3	Global pointer	yes
x4 (tp)	4	Thread pointer	yes
x5-x7	5-7	Temporaries	no
x8-x9	8-9	Saved	yes
x10-x17	10-17	Arguments/results	no
x18-x27	18-27	Saved	yes
x28-x31	28-31	Temporaries	no

Eksempel: Compiler Explorer

- Vi bruker Compiler Explorer
 - <https://godbolt.org/>



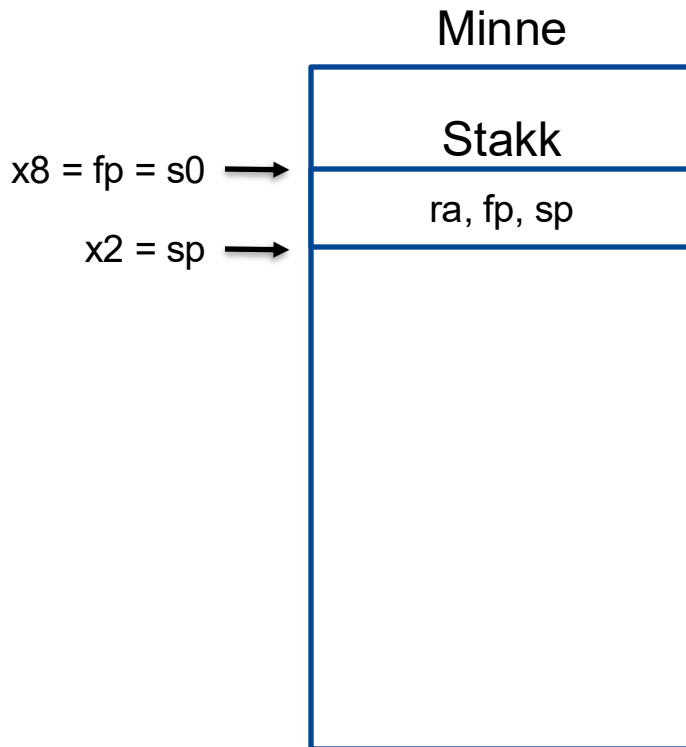
Eksempel: Stakk og rekursjon

```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```



Utverdier:

```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```



Eksempel: Stakk og rekursjon



```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

Utverdier:

```
● jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne

Stakk

ra, fp, sp

ra, fp, sp

x10 = a1 = 0 (val)

x11 = a2 = 2 (max)

x8 = fp = s0 →

x2 = sp →

Eksempel: Stakk og rekursjon

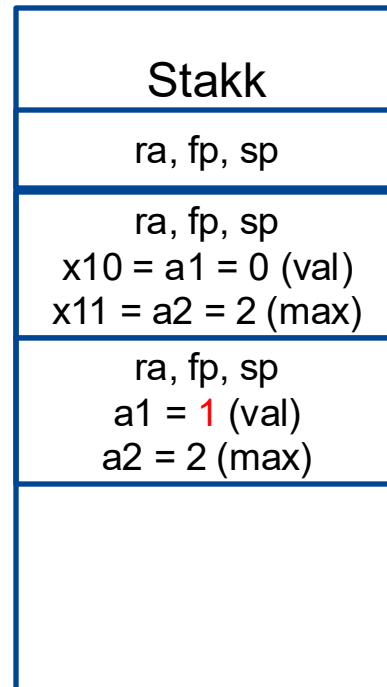


```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9          val,
10         partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

Utverdier:

```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne



x8 = fp = s0 →

x2 = sp →

Eksempel: Stakk og rekursjon



```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

Utverdier:

```
● jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne

Stakk
ra, fp, sp
ra, fp, sp x10 = a1 = 0 (val) x11 = a2 = 2 (max)
ra, fp, sp a1 = 1 (val) a2 = 2 (max)
ra, fp, sp a1 = 2 (val) a2 = 2 (max)

x8 = fp = s0 →

x2 = sp →

val == max: Vi returnerer max (2)

Eksempel: Stakk og rekursjon



```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

Utverdier:

```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne

Stakk

ra, fp, sp

ra, fp, sp

x10 = a1 = 0 (val)

x11 = a2 = 2 (max)

ra, fp, sp

a1 = 1 (val)

a2 = 2 (max)

ra, fp, sp

a1 = 2 (val)

a2 = 2 (max)

x8 = fp = s0 →

x2 = sp →

Eksempel: Stakk og rekursjon

```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

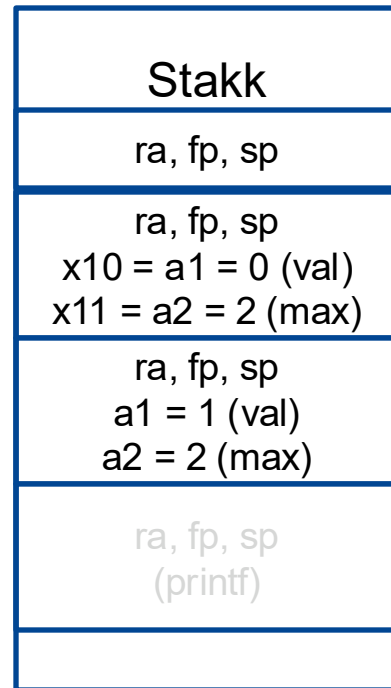


Utverdier:



```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne



x8 = fp = s0 →

x2 = sp →

val = 1, partial_sum = 2, vi returnerer 3

Eksempel: Stakk og rekursjon



```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

Utverdier:

```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne

Stakk

ra, fp, sp

x8 = fp = s0 →

ra, fp, sp

x10 = a1 = 0 (val)

x11 = a2 = 2 (max)

x2 = sp →

ra, fp, sp

a1 = 1 (val)

a2 = 2 (max)

ra, fp, sp

(printf)

Eksempel: Stakk og rekursjon

```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```

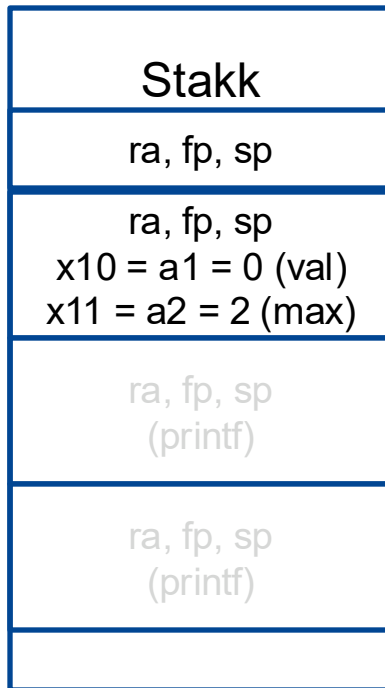


Utverdier:

```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```



Minne



val = 0, partial_sum = 3, vi returnerer 3

Eksempel: Stakk og rekursjon

```
1  #include <stdio.h>
2
3  int rekursjon(int val, int max){
4      if(val == max){
5          return max;
6      }
7      int partial_sum = rekursjon(val+1, max);
8      printf("Rekursjon: val=%d, partial_sum=%d\n",
9             val,
10            partial_sum);
11     return val + partial_sum;
12 }
13
14 int main(){
15     int retval = rekursjon(0, 2);
16     return 0;
17 }
```



Utverdier:

```
• jahre@dhcp-10-24-35-176 F3 % ./rekursjon
Rekursjon: val=1, partial_sum=2
Rekursjon: val=0, partial_sum=3
```

Minne

Stakk

x8 = fp = s0 →

ra, fp, sp

x2 = sp →

ra, fp, sp

x10 = a1 = 0 (val)

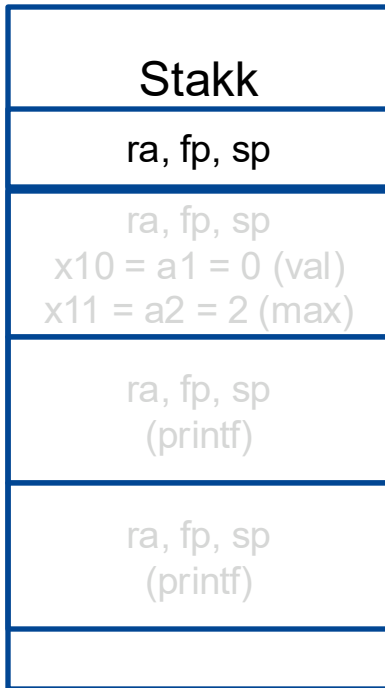
x11 = a2 = 2 (max)

ra, fp, sp

(printf)

ra, fp, sp

(printf)





Tema 2.4: Instruksjoner, diverse (Kapittel 2.9 - 2.12)

TEKST, STORE KONSTANTER, ADRESSEMODI, SYNKRONISERING, OG OPPSTART AV PROGRAMMER

Læringsutbytte T2.4

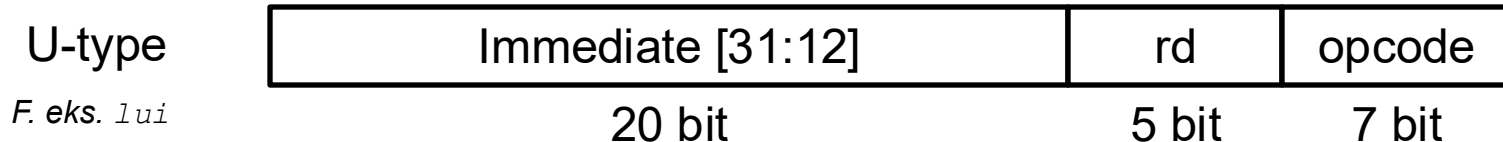
- Studenten kjenner til hvordan **tekst** representeres i en datamaskin.
- Studenten skal vite hvordan vi håndterer **store konstanter** og (unngår) **lange hopp** (PC-relativ adressering).
- Studenten skal kunne forklare RISC-Vs fire **adressemodi**.
- Studenten skal kunne forklare skrittene involvert i **oversettelse og oppstart av programmer** (kunne gjengi og forklare figur 2.20)

Bokstaver representeres som tall

- Som alt annet i datamaskinen representerer vi bokstaver som tall
 - ... men hvor mange bit trenger vi?
 - Ikke 32 for vi trenger ikke å representere 2^{32} bokstaver (~4.3 milliarder)
- To vanlige representasjoner
 - ASCII: 8 bit per bokstav (en byte)
 - Unicode: Typisk 16 bit per bokstav (2 byte eller et halvt ord, «halfword»)
- Bokstaver er vanlige og RISC-V har derfor egne instruksjoner for byte og halvord:
 - lbu – «load byte unsigned»
 - lhu – «load halfword unsigned»

ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character	ASCII value	Character
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Store konstanter



- Hva gjør vi hvis vi har konstanter som ikke kan representeres med 20 bit?

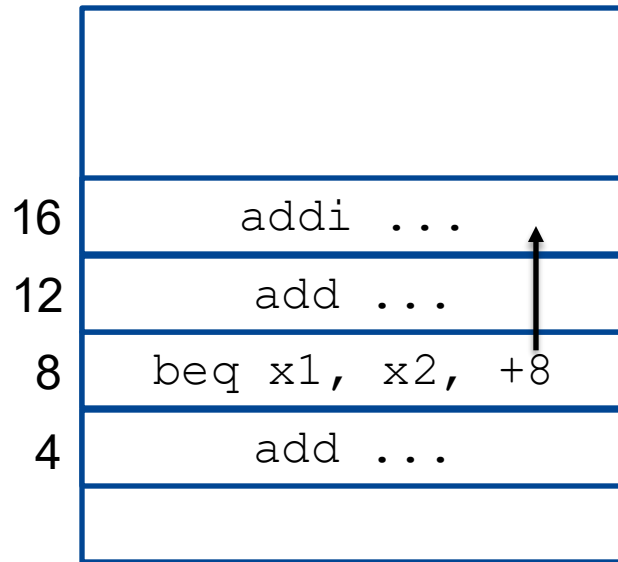
Verdien i x1

```
lui x1, 0xababa      1010 1011 1010 1011 1010 0000 0000 0000
addi x1, x1, 0x789    + 0000 0000 0000 0000 0000 0111 1000 1001
                     = 1010 1011 1010 1011 1010 0111 1000 1001
```

Hvordan (unngå å) hoppe langt

- Programmet kan bruke hele det 32-bit store adresseområdet
 - ... så alle statisk definerte adresser er «store konstanter»
- De fleste adresser vi hopper til er statisk definerte
 - ... og det er ikke effektivt å bruke to instruksjoner på hver forgreningsinstruksjon
- Løsning 1: PC-relativ adressering
 - Angi måladressen til forgreningsinstruksjoner relativ til den nåværende instruksjonsadressen
 - Avstand kan være både positiv og negativ
 - Husk: «Program Counter (PC)» er et spesielt register som lagrer den nåværende instruksjonsadressen
- Løsning 2: `auipc rd, imm`
 - Skifter konstanten til de mest signifikante 20 bitene og setter de resterende bitene til 0
 - Legger dette til PC og lagrer resultatet i destinasjonsregisteret
 - Gjør det mulig å hoppe hvor som helst i adresserommet
 - Bonus: Hvis `imm` er 0, lagres PC i `rd`

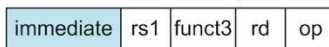
Minne



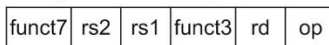
Adressemodi

- Måten vi kan oppgi en adresse på kalles en adressemodus
- RISC-V støtter fire ulike adressemodi
 - Konstantbasert adressering
 - Registerbasert adressering
 - Grunnadressebasert adressering
 - PC-relativ adressering

1. Immediate addressing



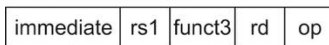
2. Register addressing



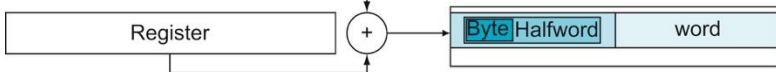
Registers

Register

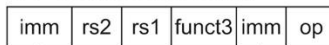
3. Base addressing



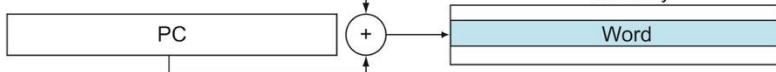
Memory



4. PC-relative addressing



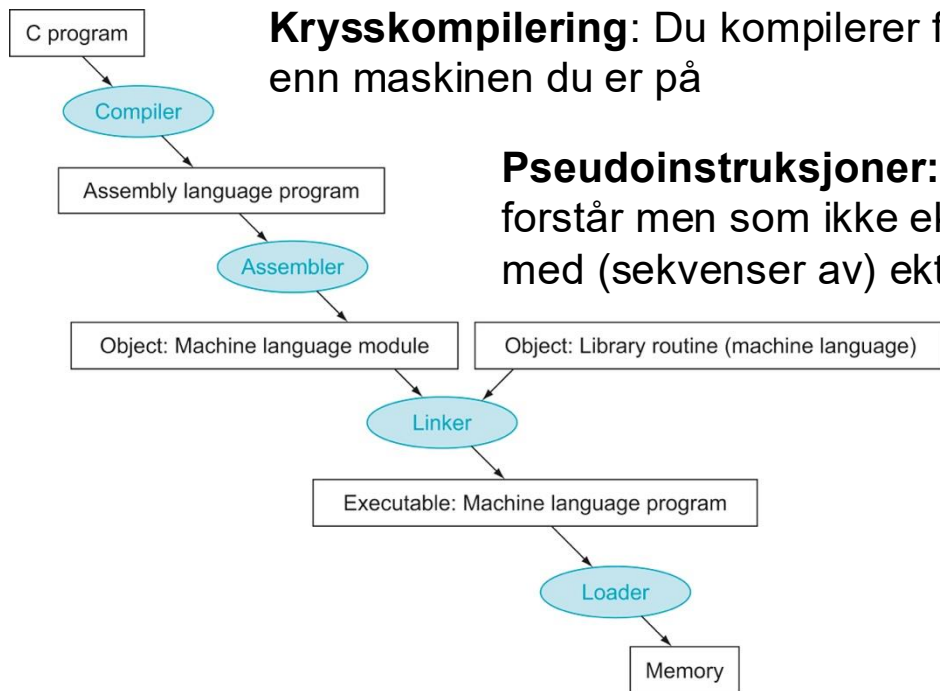
Memory



Synkroniseringsinstruksjoner

- Når vi har mer enn en prosessor, må vi av og til sikre at bare en prosessor gjør noe av gangen
 - Dette kalles synkronisering
 - .. og koden som må utføres alene er en kritisk seksjon («critical section»)
- Synkroniseringsmekanismer blir mer effektive hvis de støttes av spesialinstruksjoner
 - Fundamentalt må vi først lese og så skrive en minnelokasjon og kunne garantere at ingenting har skjedd mellom leseoperasjonen og skriveoperasjonen
 - Typisk implementert med to instruksjoner: «Load reserved» and «store conditional».

Oversettelse og oppstart av programmer



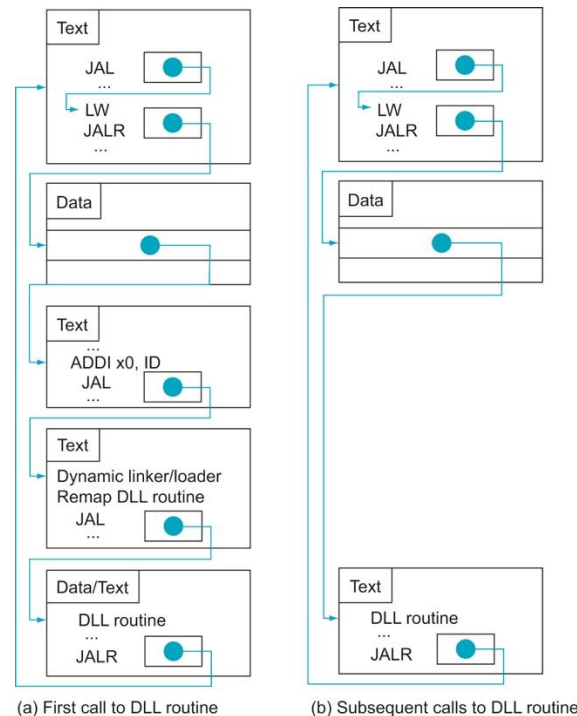
Krysskompilering: Du kompilerer for et annet instruksjonssett enn maskinen du er på

Pseudoinstruksjoner: Instruksjoner som assembleren forstår men som ikke eksisterer i instruksjonssettet. Byttes ut med (sekvenser av) ekte instruksjoner i assembleren.

Figur 2.20

Statisk og dynamisk linking

- Ved statisk linking kopierer linkereren delte bibliotek inn i en «executable»
 - ... men dette er ineffektivt fordi bibliotekene (i) ikke kan oppdateres uavhengig av programmet og (ii) biblioteksfunksjonene lastes uavhengig av om de blir brukt eller ikke
- Løsning: Dynamisk linking
 - Vi har en tabell i minnet med pekere til biblioteksfunksjonene programmet kan bruke
 - På første aksess kalles en funksjon som (i) laster inn biblioteksfunksjonen, (ii) oppdaterer pekeren til å peke dit biblioteksfunksjonen er, og (iii) kaller biblioteksfunksjonen
 - Alle senere kall går direkte fra tabellen til biblioteksfunksjonen



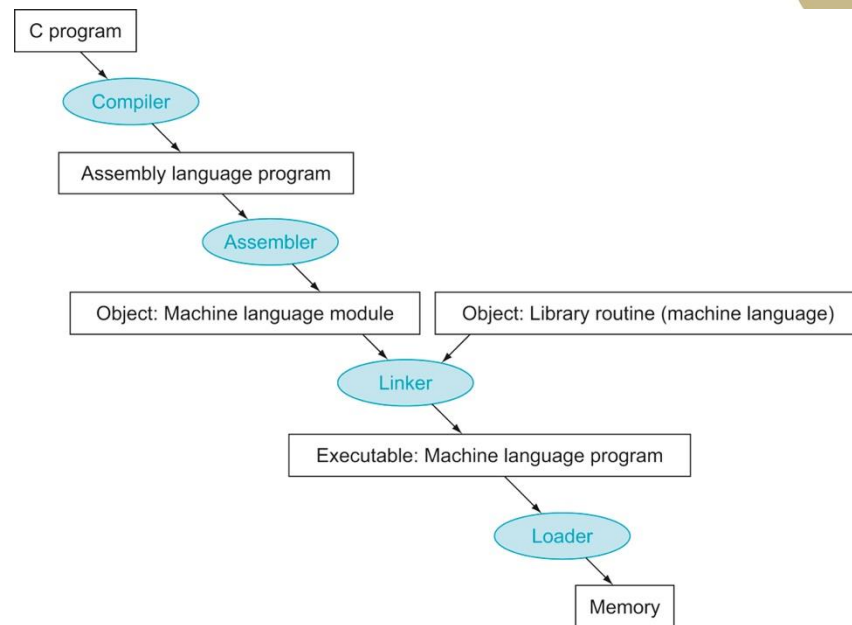


Tema 2: Oppsummering (Kapittel 2.22)

OPPSUMMERING

Assembly er «raskt»

- Man skulle tro at man får bedre ytelse av å skrive assembly selv
 - ... men det gjør man (typisk) ikke
- Grunnene er at kompilatorer er utrolig gode og datamaskiner er voldsomt kompliserte



Det store bildet

- Instruksjonssett designes etter følgende prinsipper
 - Regelmessighet forenkler
 - Mindre er raskere
 - Gode løsninger krever gode kompromiss
- Instruksjonssettet er en av mange abstraksjoner
 - Vi håndterer kompleksitet gjennom å gjemme bort detaljer
 - Å kunne forholde seg til abstraksjoner er nøkkelen til å forstå datamaskiner

